

LAB 2 - Intro to Prometheus and Grafana

After downloading and unzipping the file, made the necessary changes to the docker compose file and started the containers with **docker compose up -d**

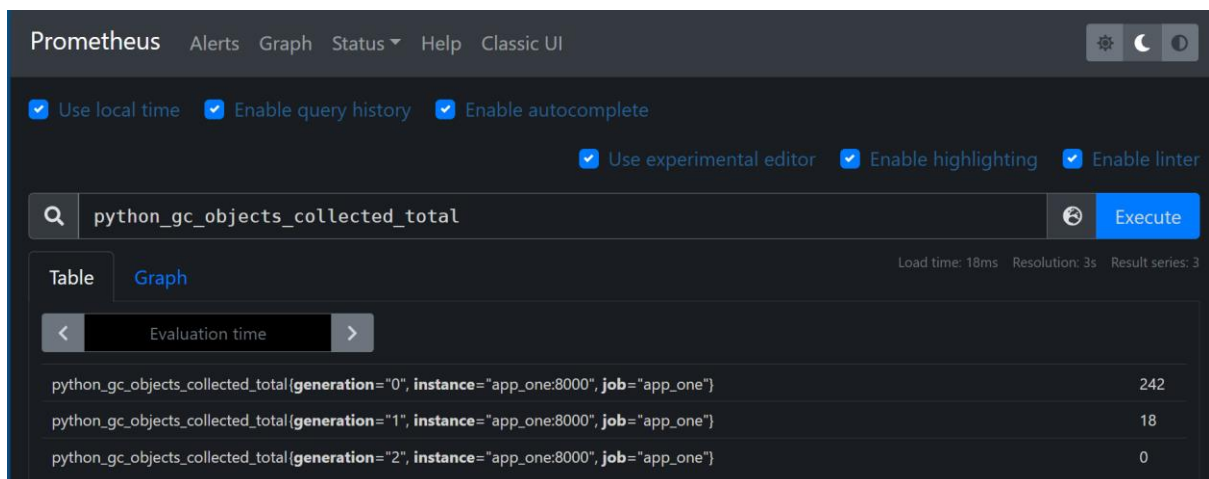
```
oadevusi@oluSec:~/aiops/Assignments/LAB2-Prometheus_Grafana$ docker compose up -d
[+] Running 8/8
 ✓ Container prometheus_push_gateway Started
 ✓ Container app_one Started
 ✓ Container prometheus_node_exporter Started
 ✓ Container grafana Started
 ✓ Container prometheus Started
 ✓ Container app_two Started
 ✓ Container postgres Started
 ✓ Container postgres_exporter Started
```

Began with python metrics with app1

For APP1:

Python metrics:

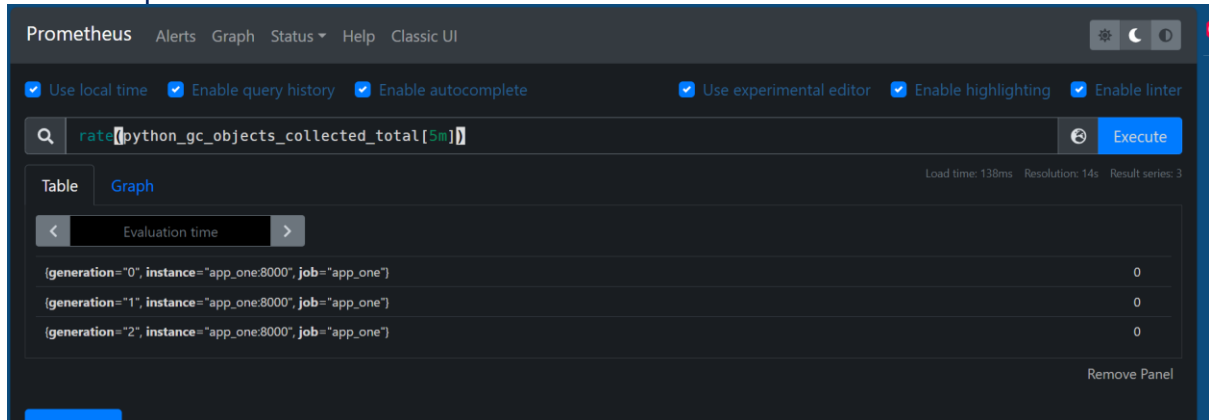
python_gc_objects_collected_total



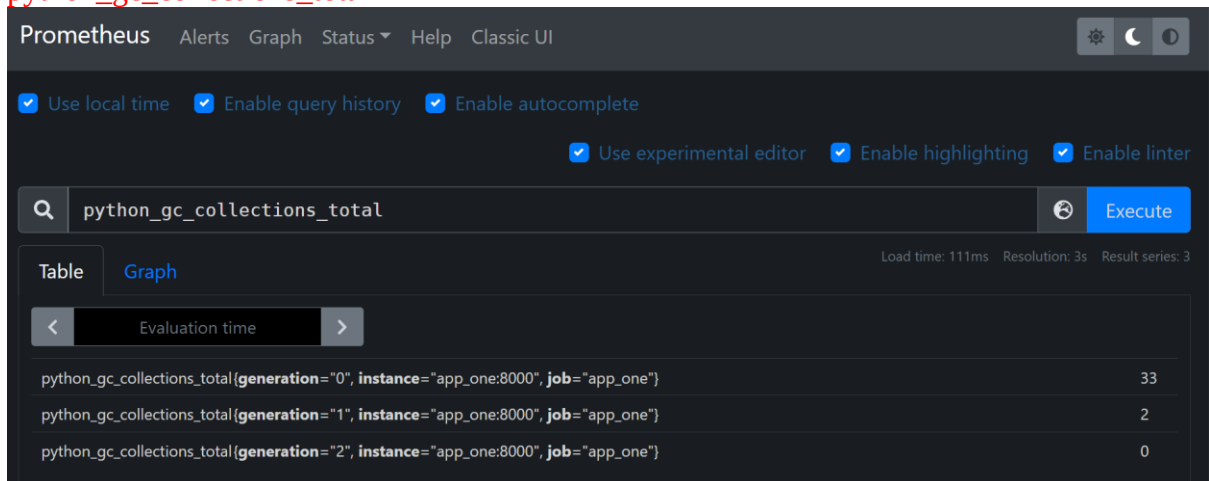
Visualization on Grafana dashboard:



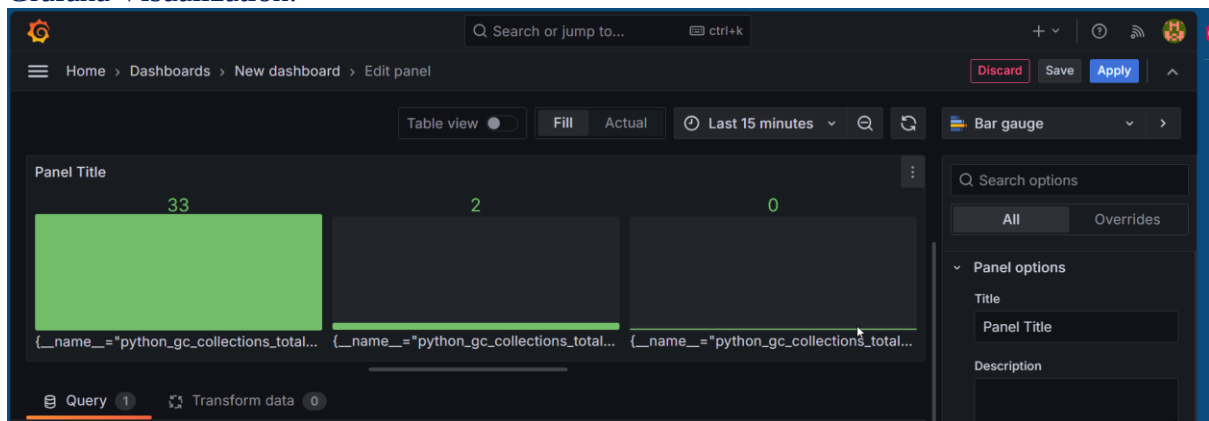
Used the operator “rate” and “time” on the same metrics



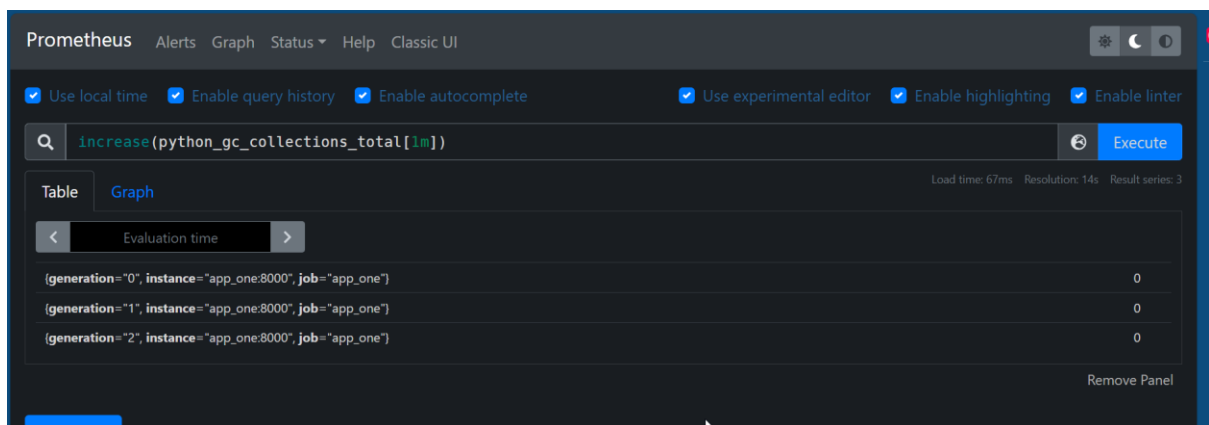
Another python metrics:
python_gc_collections_total



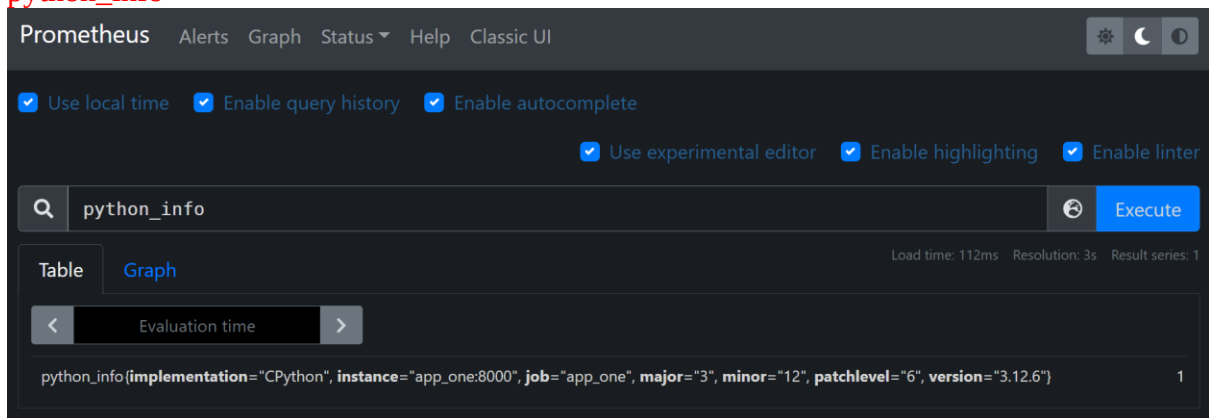
Grafana Visualization:



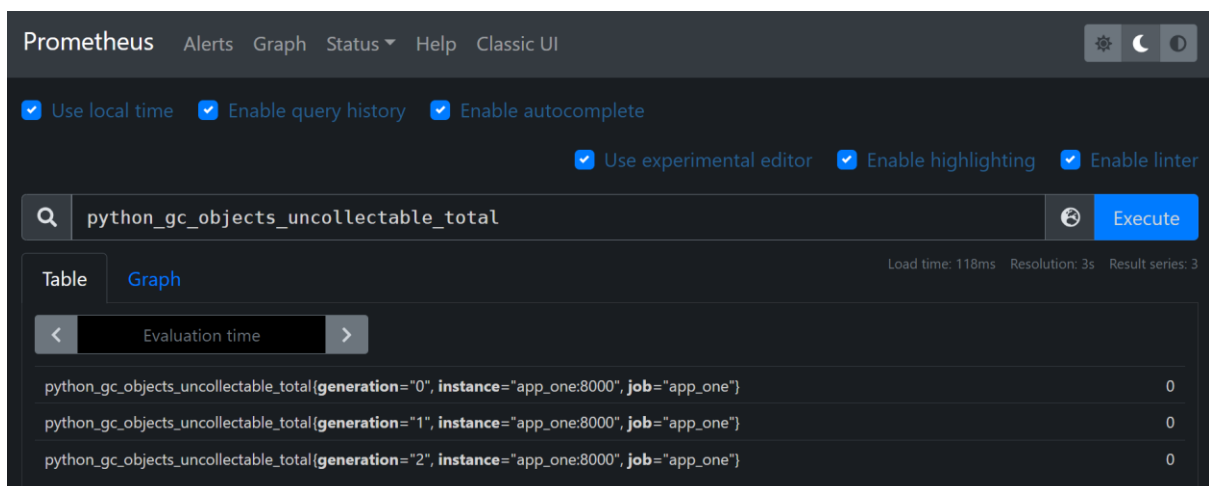
Used the operators “increase” and “time”:



Another python metrics:
`python_info`



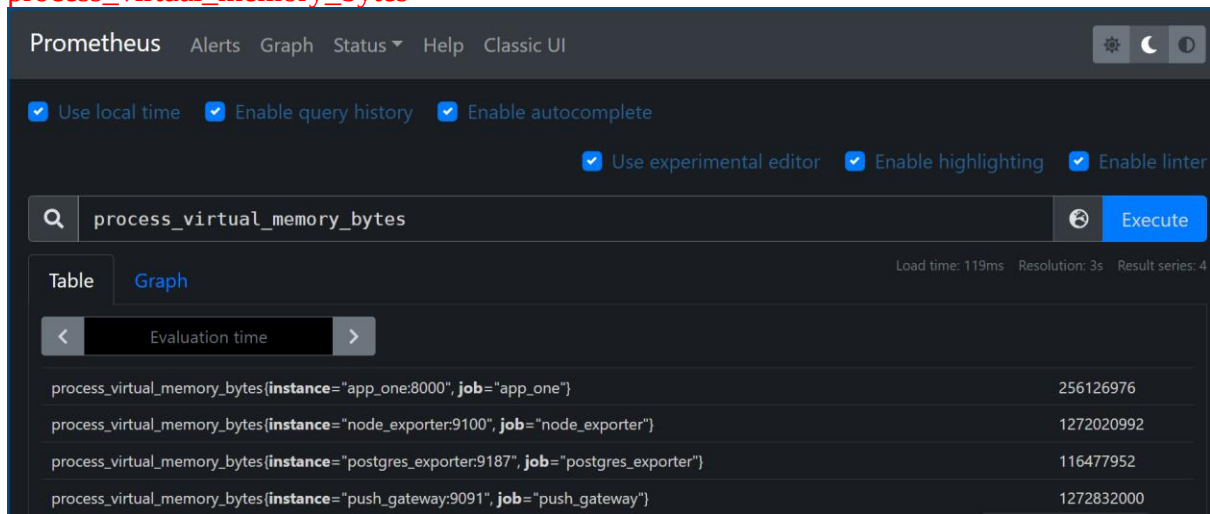
Python metrics:
`python_gc_objects_uncollectable_total`



Displayed some of the system metrics:

Linux/system metrics:

process_virtual_memory_bytes

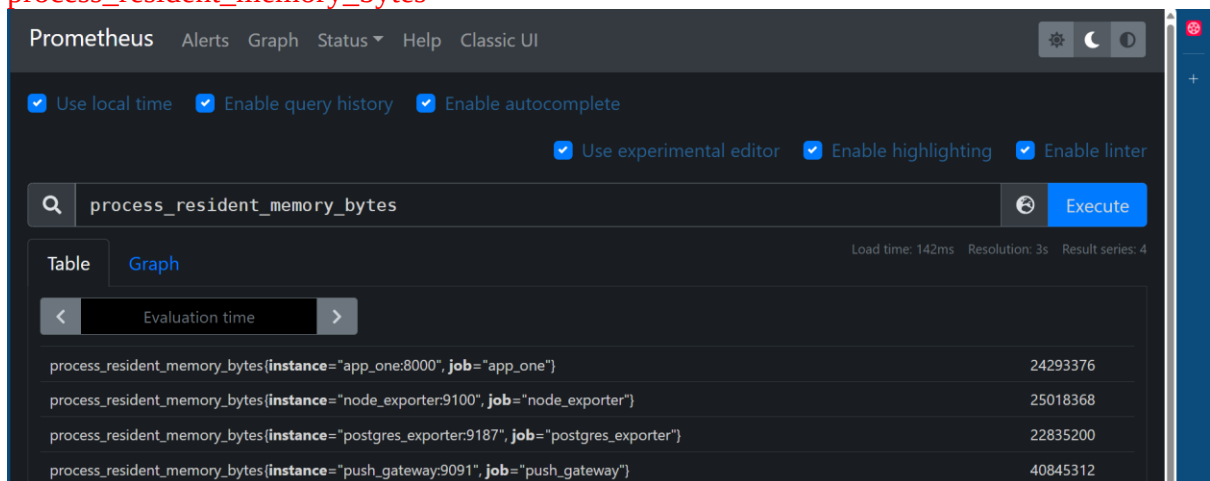


Using time operator:

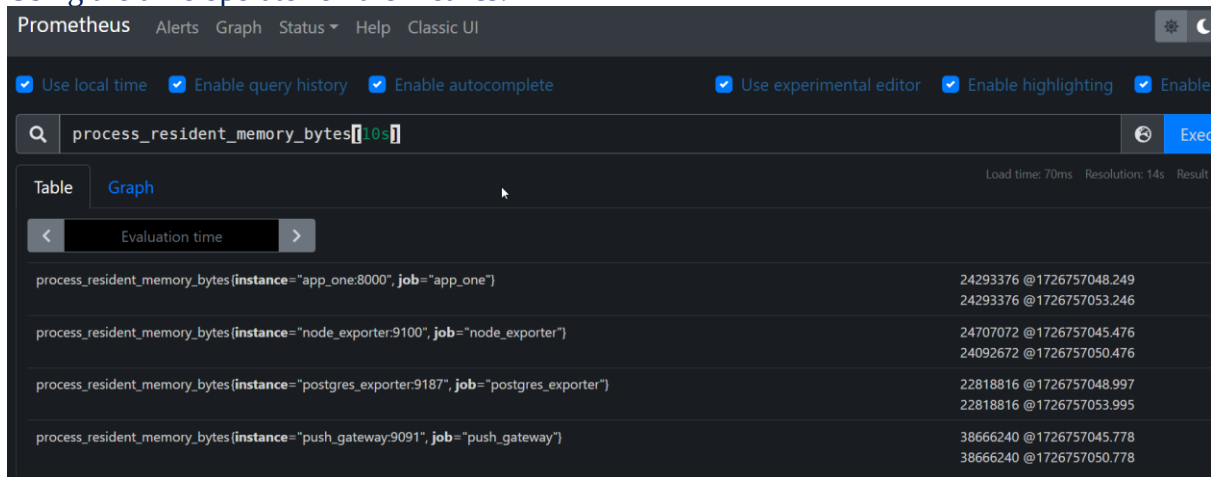


Another system metrics:

process_resident_memory_bytes

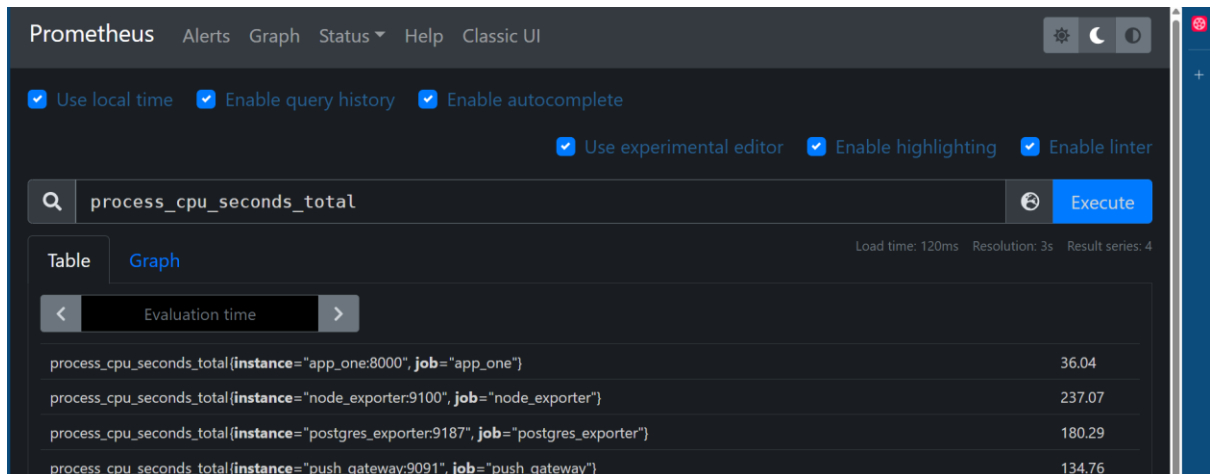


Using the time operator on the metrics:

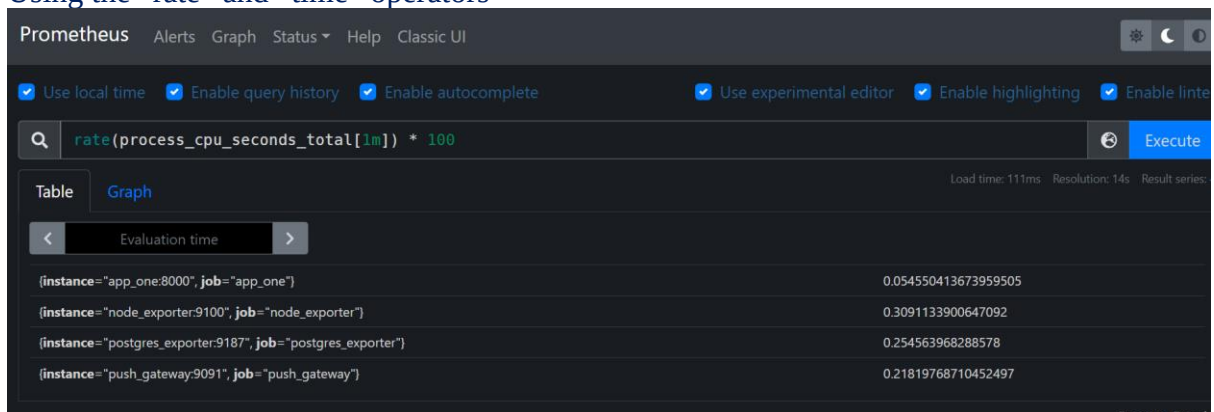


System metrics:

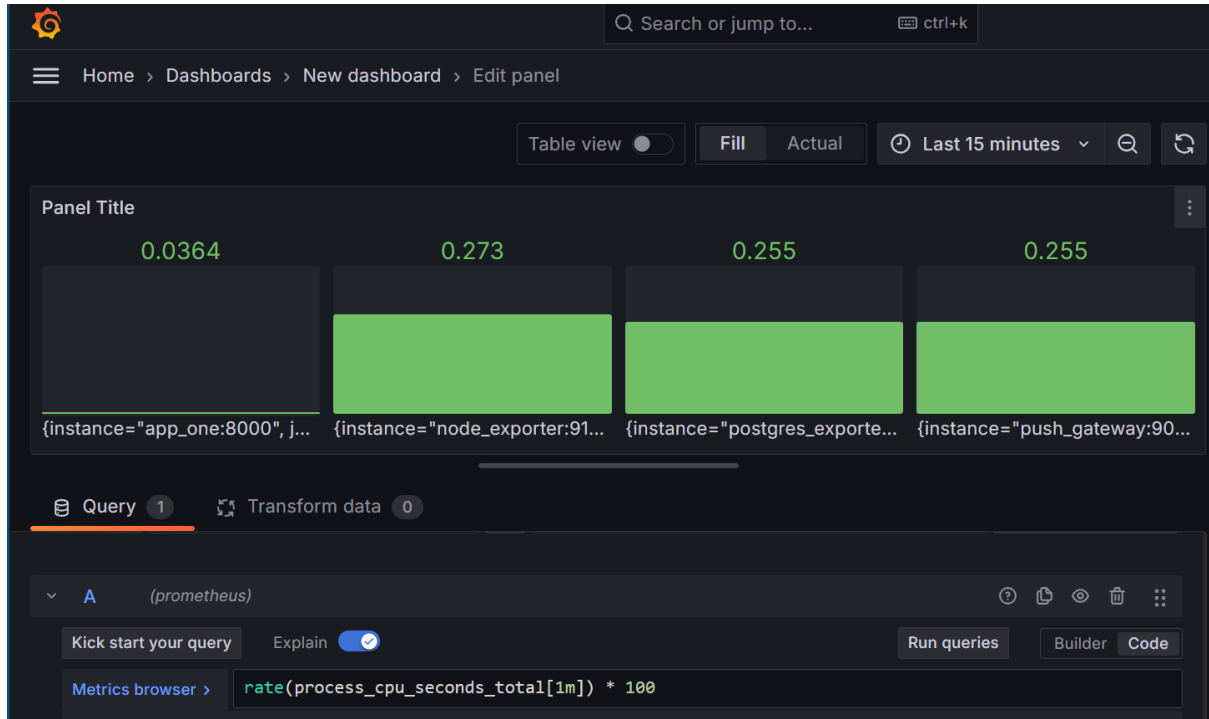
`process_cpu_seconds_total`



Using the “rate” and “time” operators



Grafana visualization:



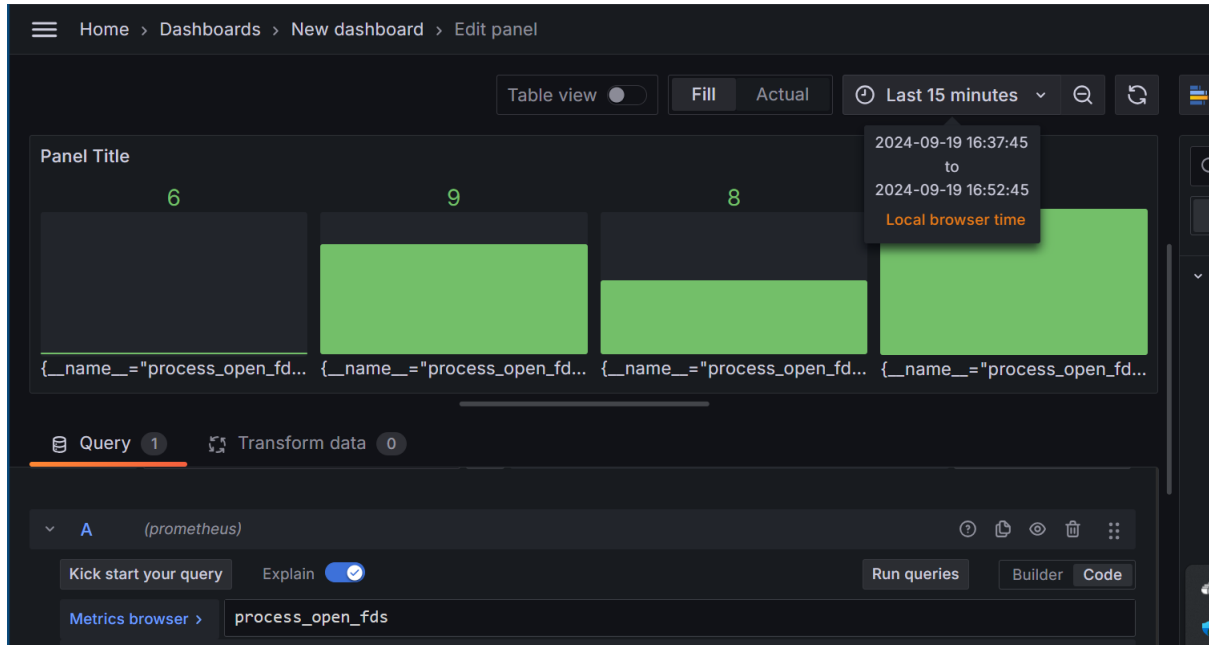
System metrics:

process_open_fds

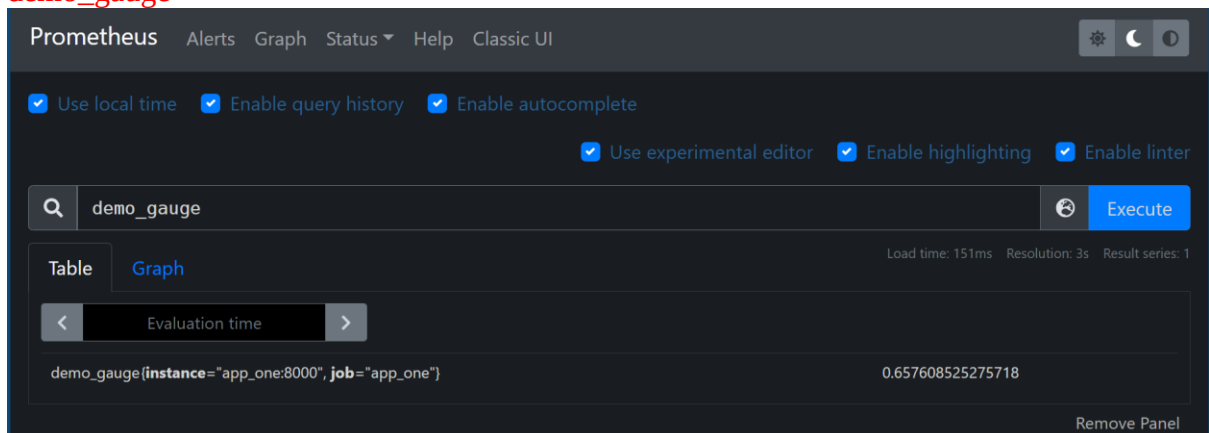
The image shows the Prometheus web interface. The query `process_open_fds` has been executed, and the results are displayed in a table. The table has two columns: the metric name and the value. The values are 6, 9, 8, and 10.

Metric	Value
process_open_fds(instance="app_one:8000", job="app_one")	6
process_open_fds(instance="node_exporter:9100", job="node_exporter")	9
process_open_fds(instance="postgres_exporter:9187", job="postgres_exporter")	8
process_open_fds(instance="push_gateway:9091", job="push_gateway")	10

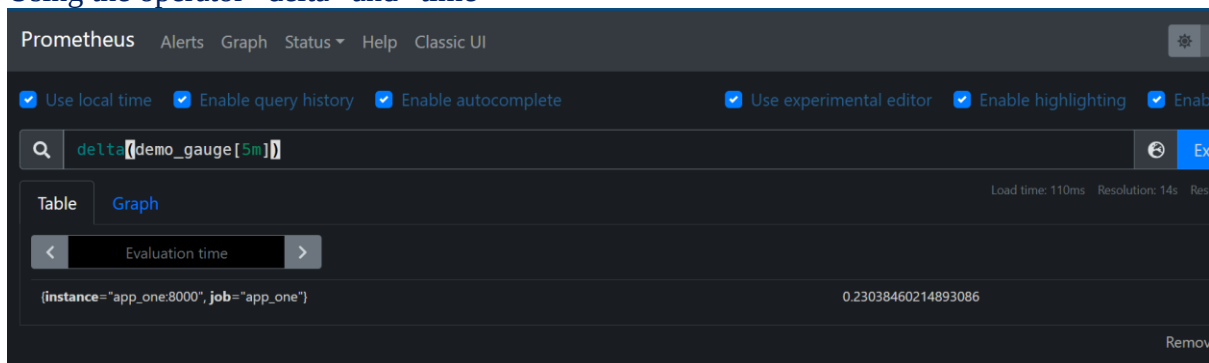
Grafana visualization:



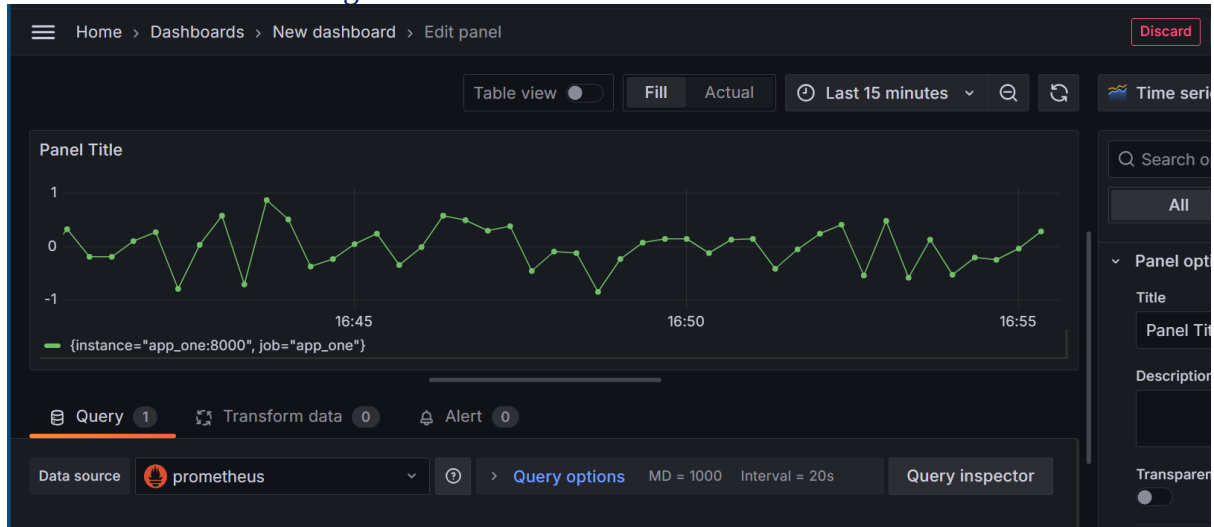
Application-specific metrics: demo_gauge



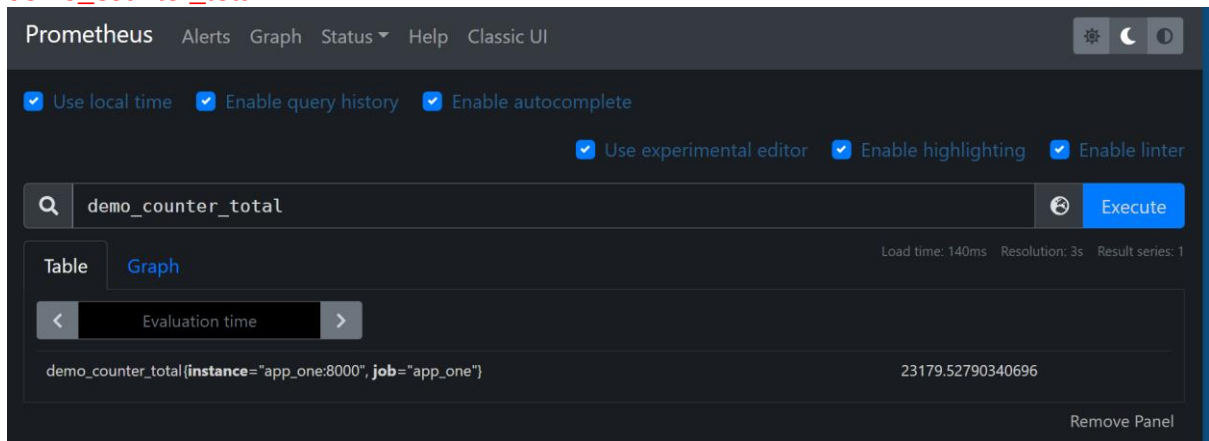
Using the operator “delta” and “time”



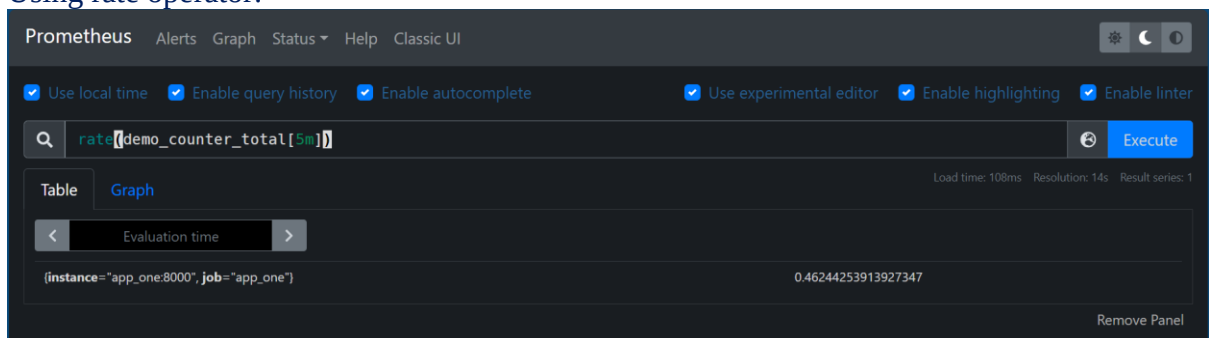
Grafana visualization using time series visuals:



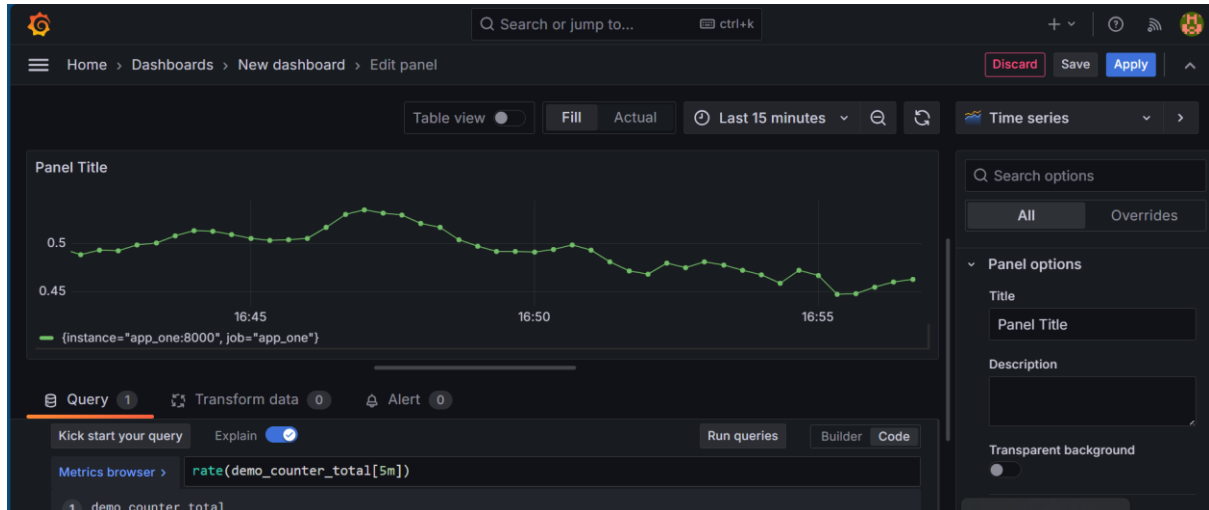
Another app1 metric: demo_counter_total



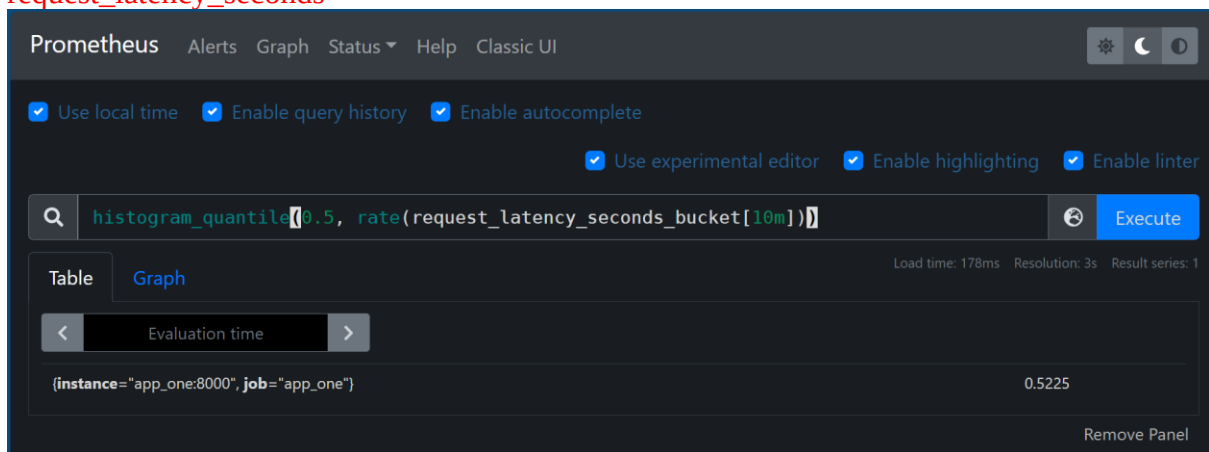
Using rate operator:



Grafana visualization:



App1 metric using histogram to find the 50th percentile:
request_latency_seconds

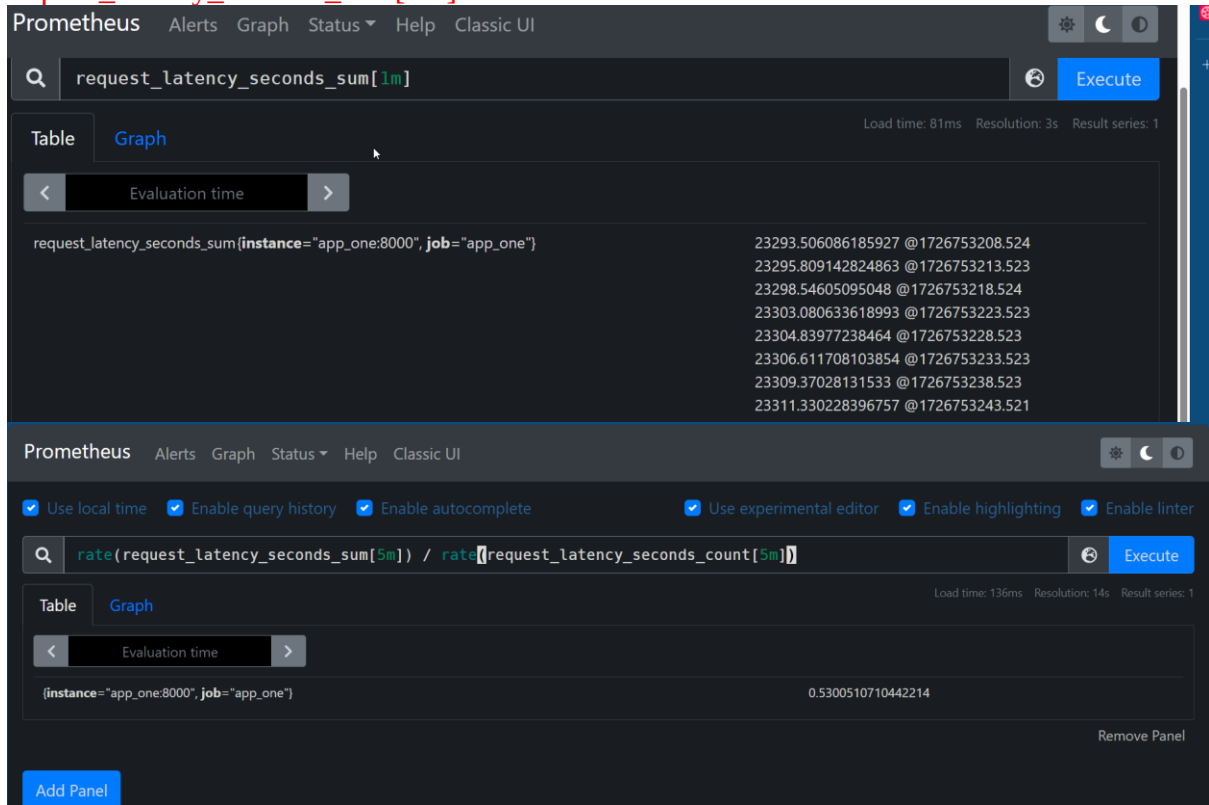


Grafana Visuals:

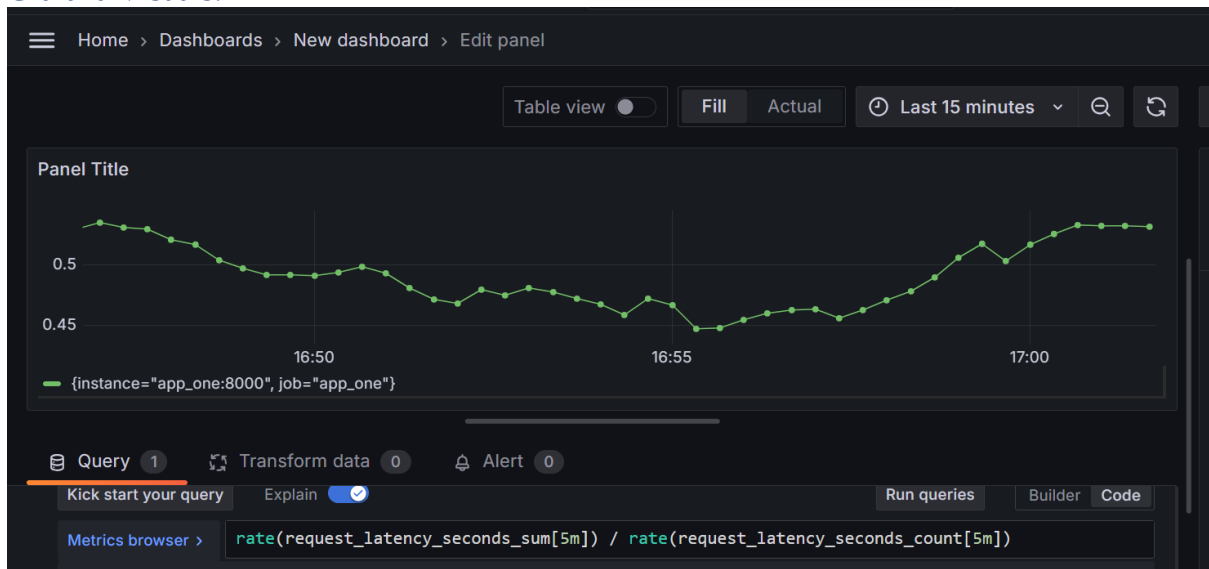


Another app specific metric:

`request_latency_seconds_sum[1m]`



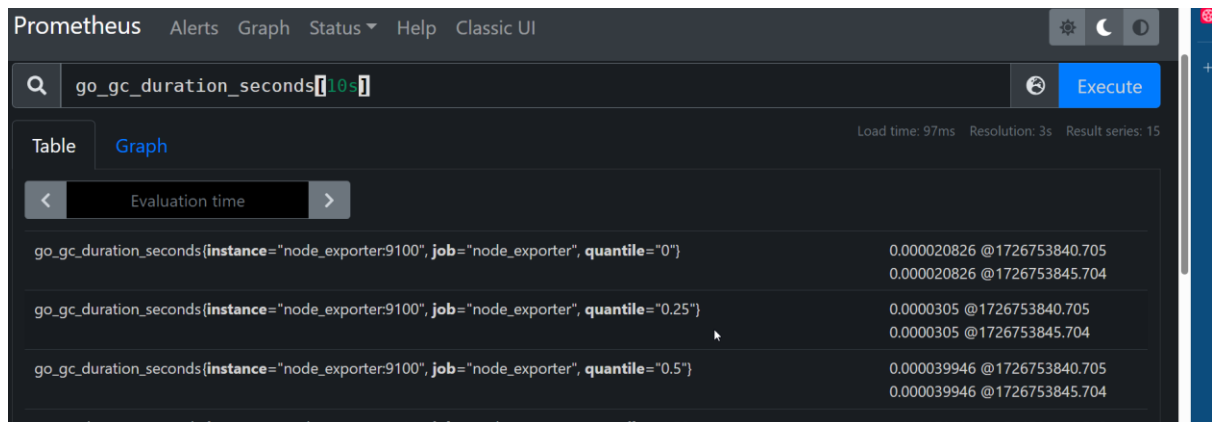
Grafana Visuals:



For APP2

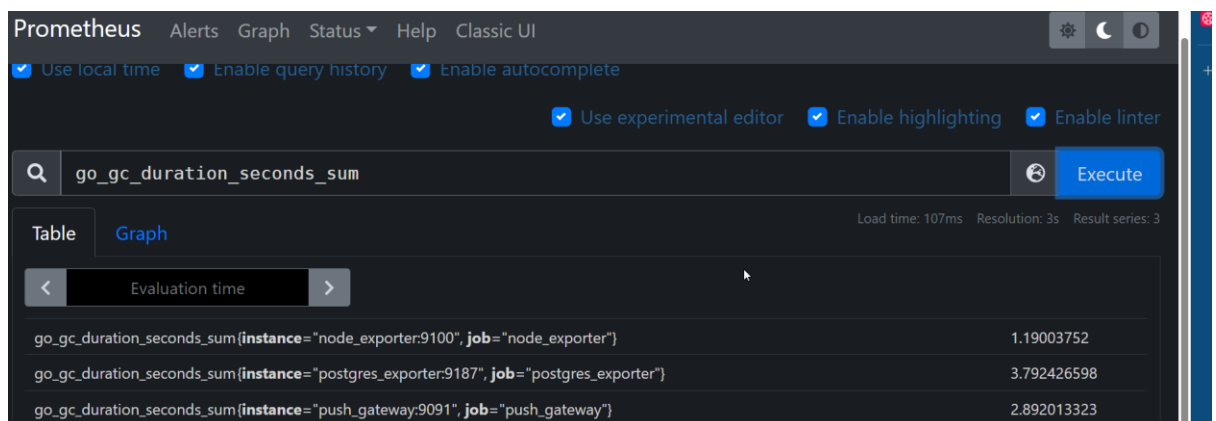
go metrics on app2:

`go_gc_duration_seconds`



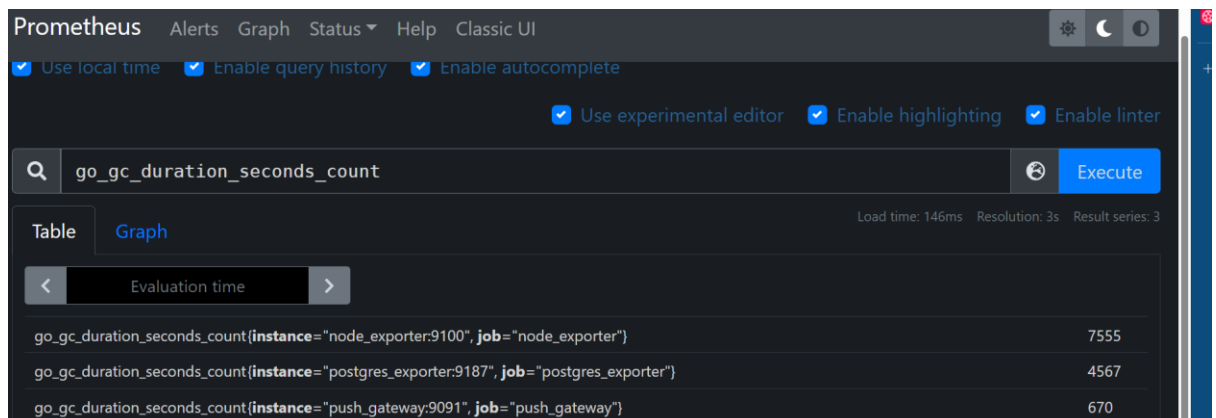
Another go metric:

`go_gc_duration_seconds_sum`

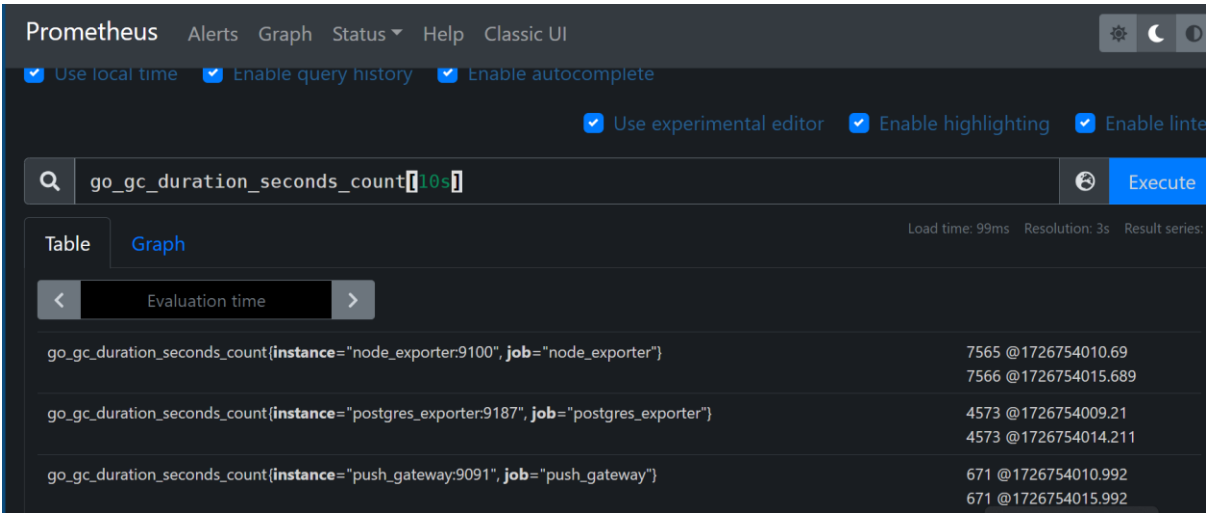


go metrics for app2:

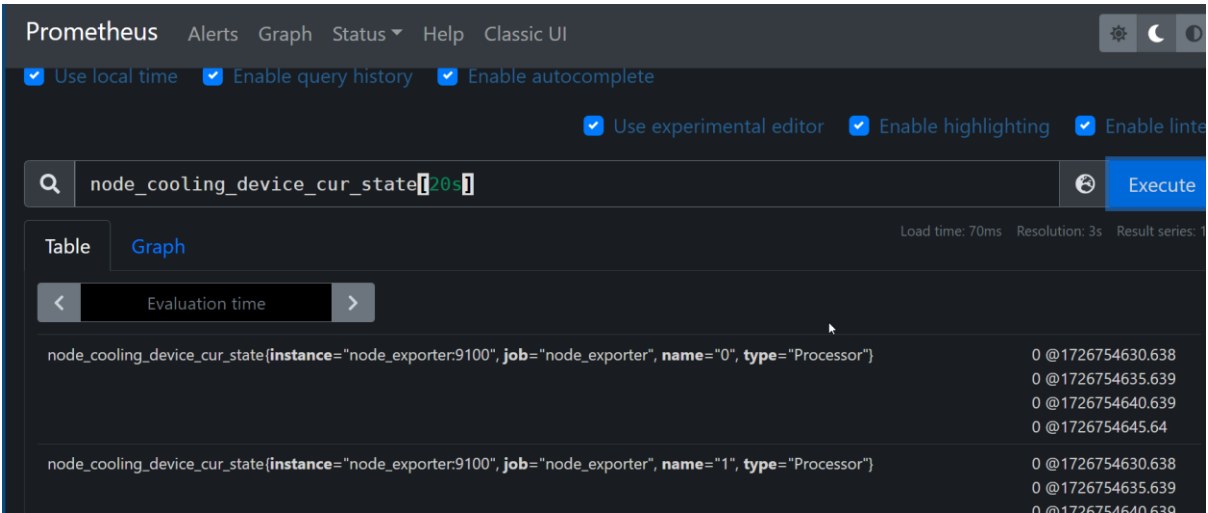
`go_gc_duration_seconds_count`



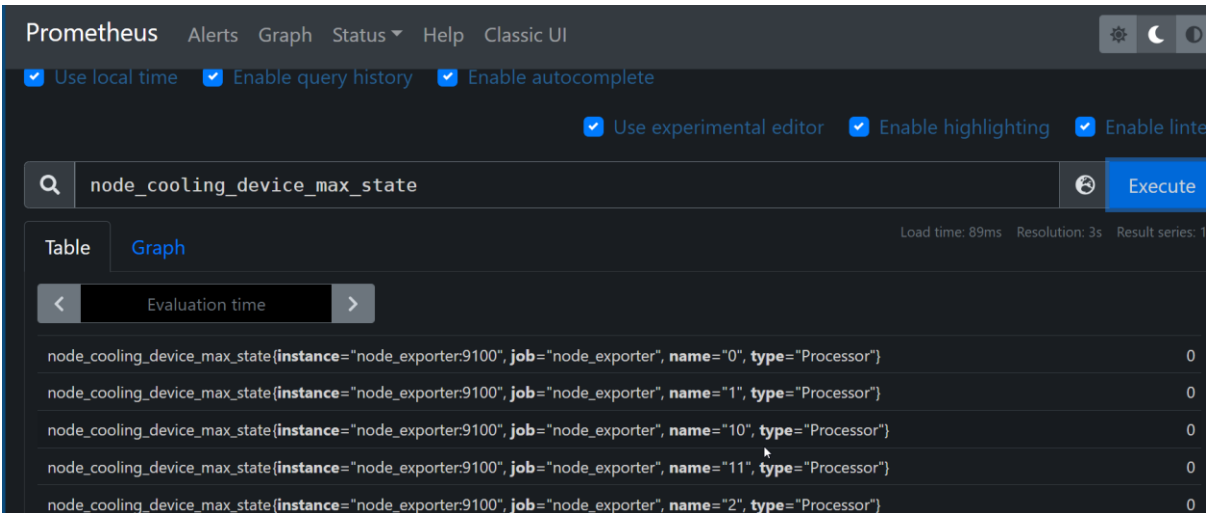
Using rate operator:



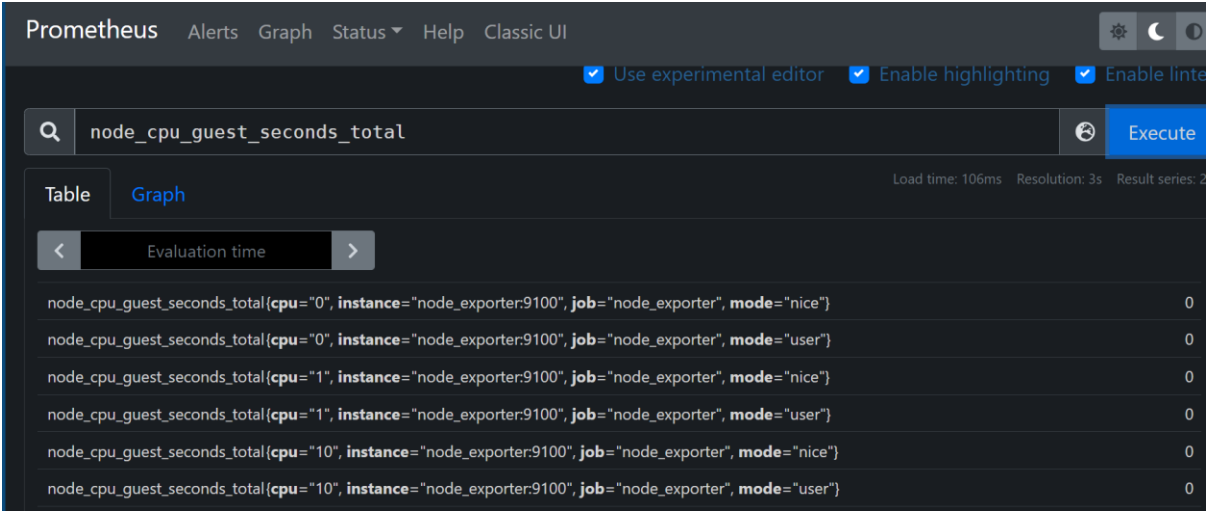
Linux/system metrics specific to app2: `node_cooling_device_cur_state`



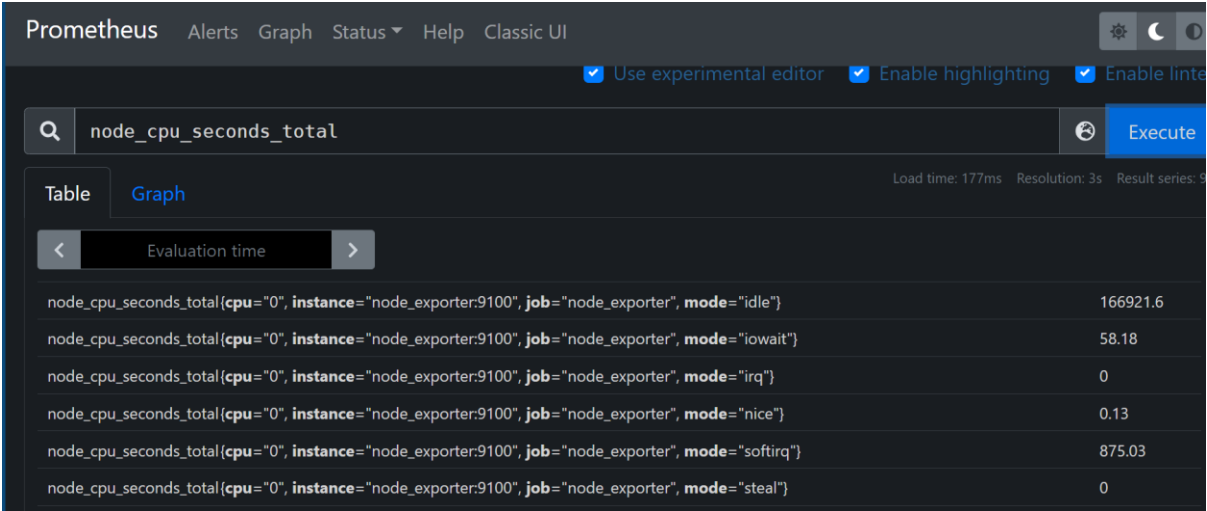
Another system metrics:
`node_cooling_device_max_state`



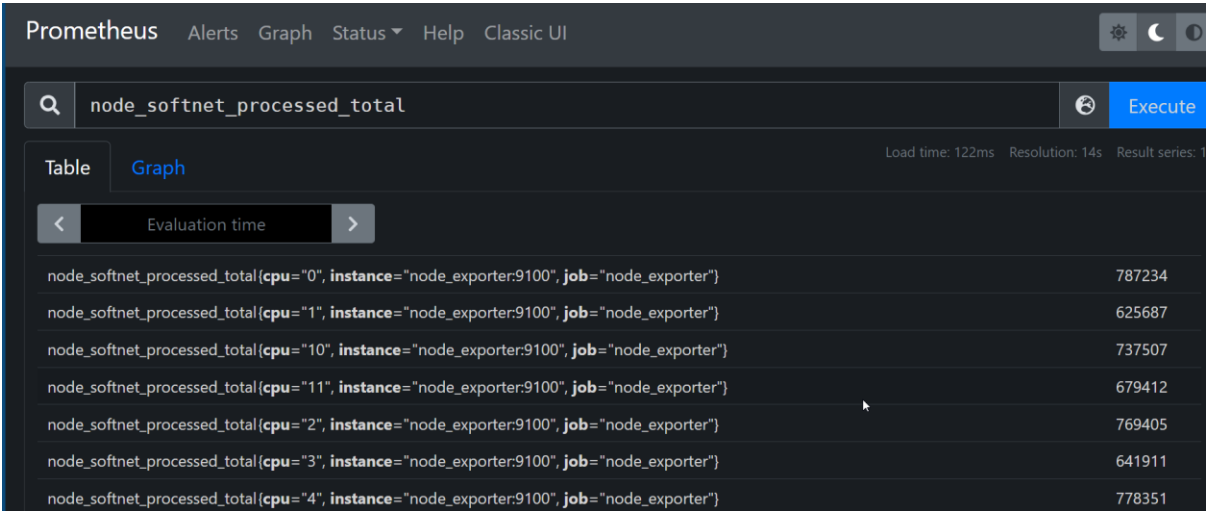
node_cpu_guest_seconds_total



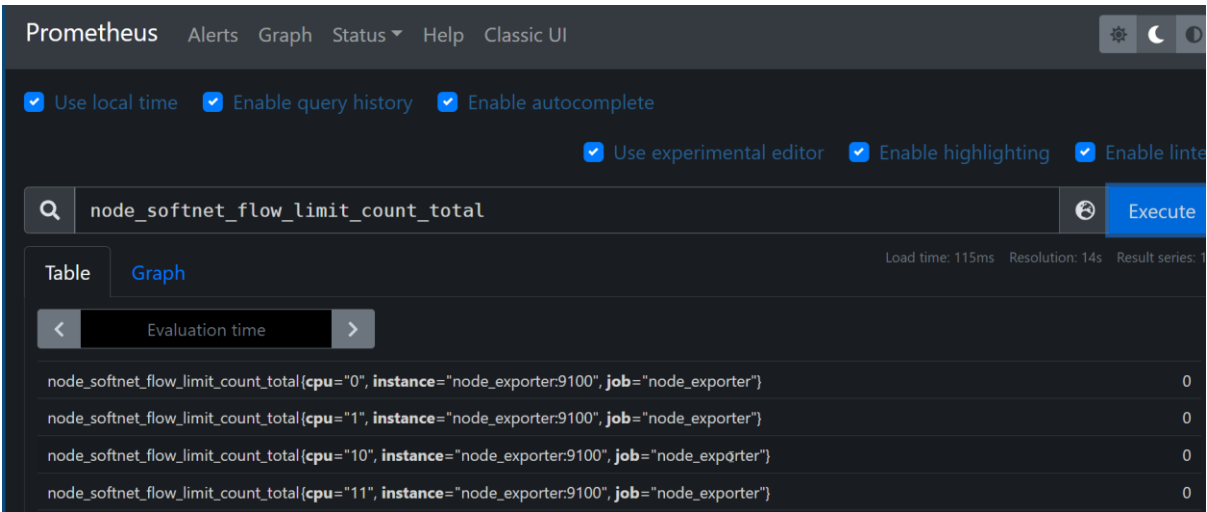
node_cpu_seconds_total



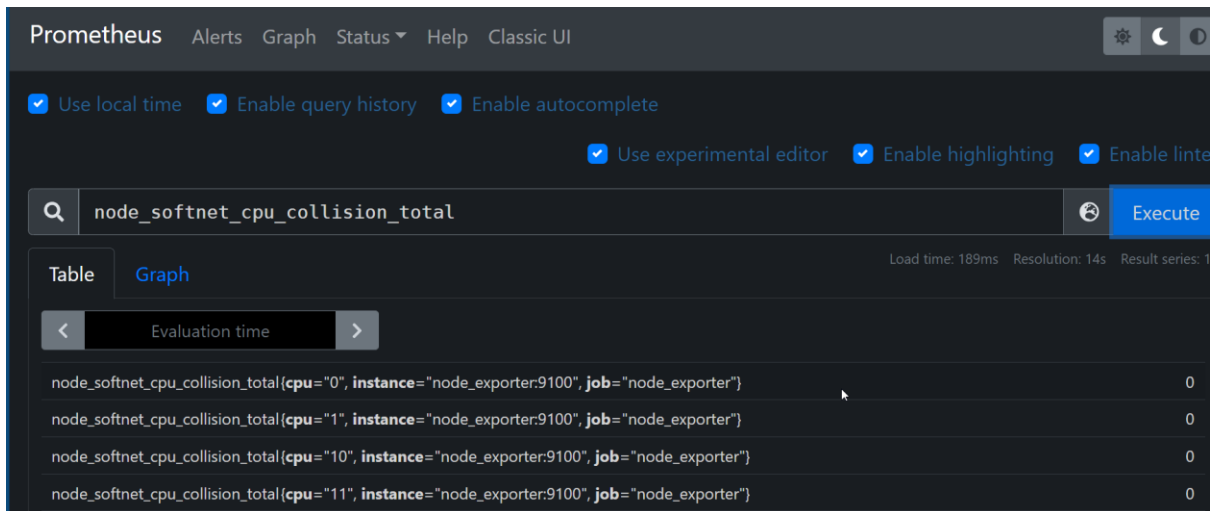
Network-related metrics on app2:
node_softnet_processed_total



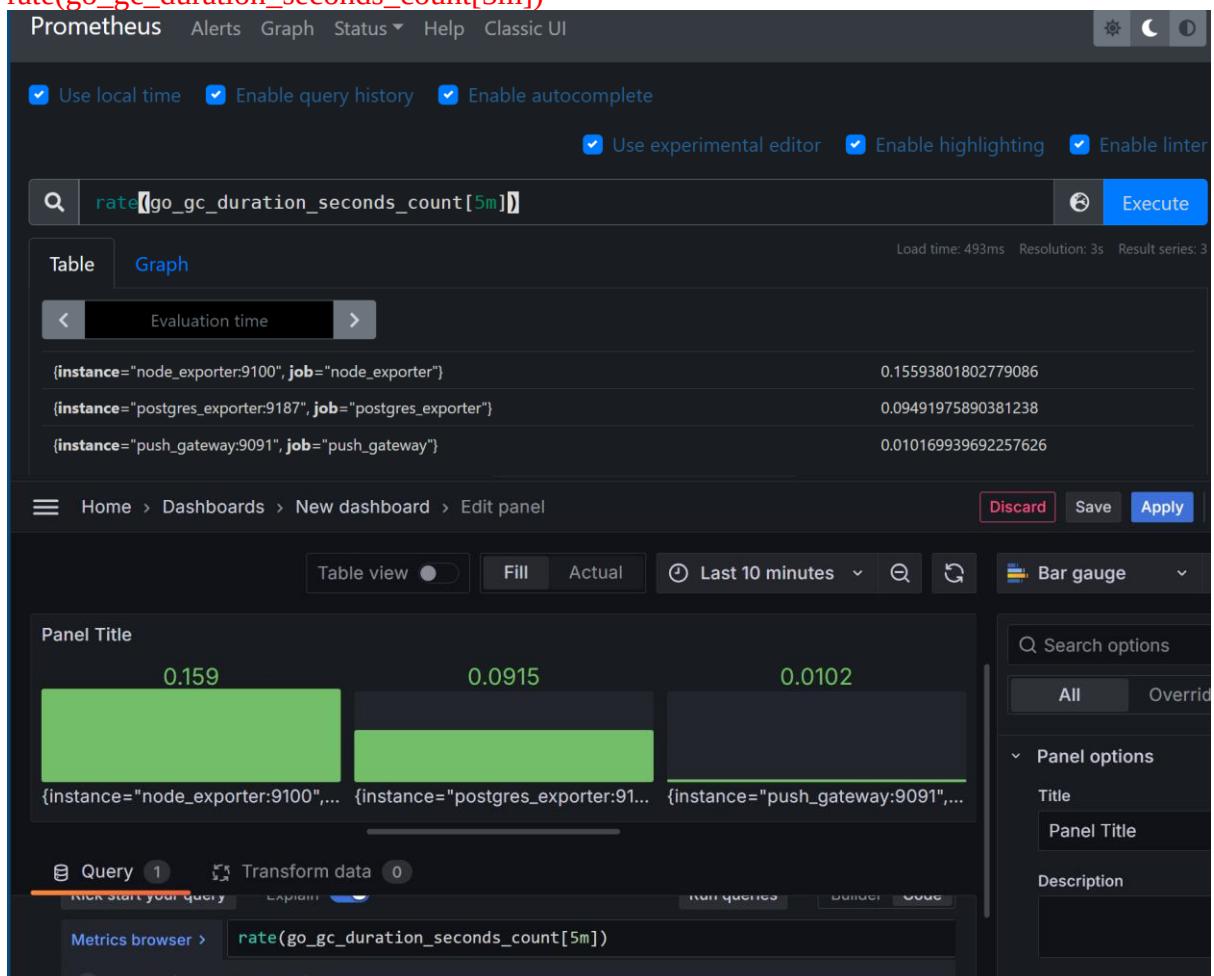
node_softnet_flow_limit_count_total



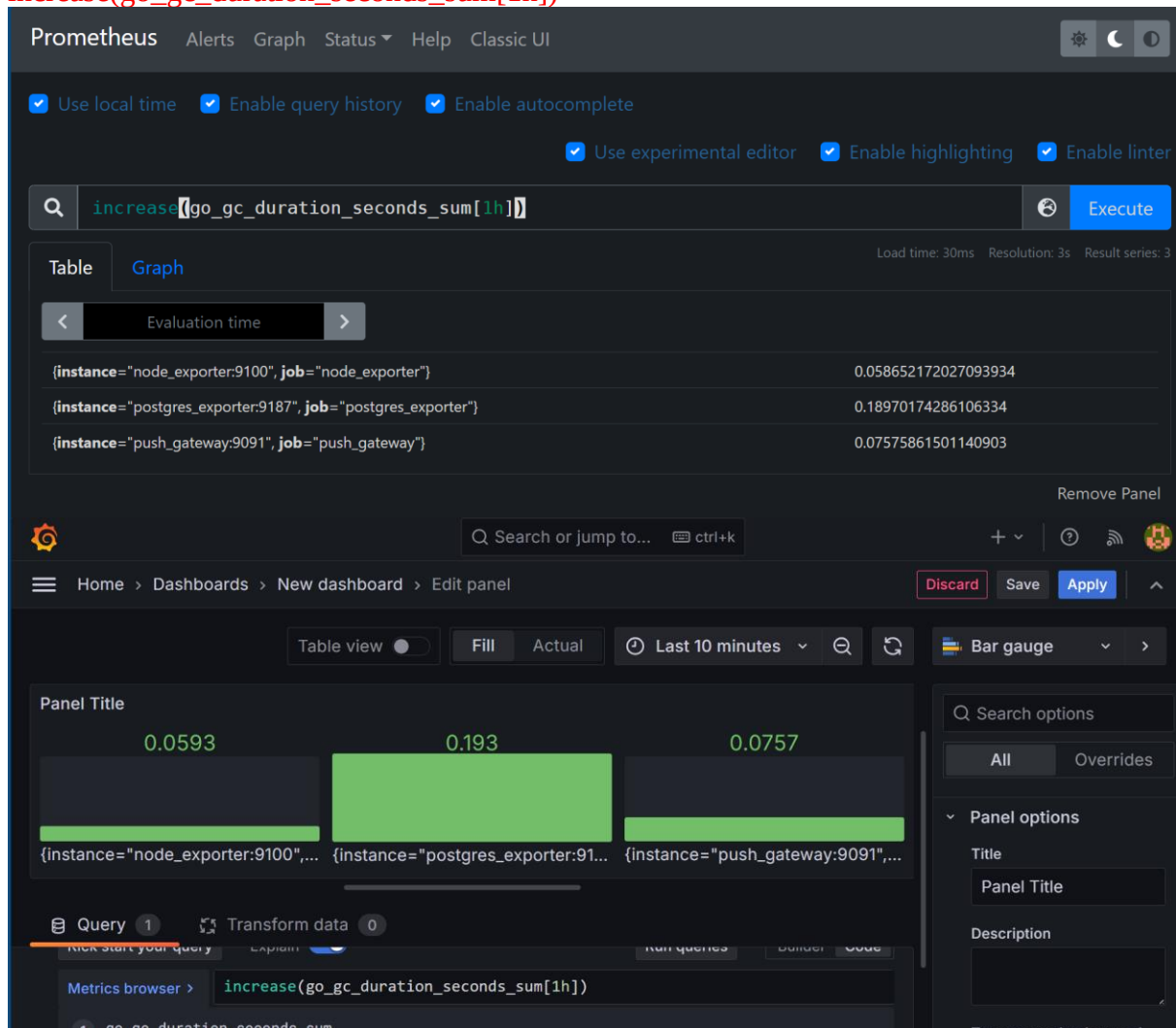
node_softnet_cpu_collision_total



rate(go_gc_duration_seconds_count[5m])



`increase(go_gc_duration_seconds_sum[1h])`



Modification of app.py for app_one:


```
app.py U X
LAB2-Prometheus_Grafana > containers > app_one > app.py
1 from prometheus_client import start_http_server, Gauge, Histogram
2 import random
3 import time
4
5 # Define metrics
6 test_gauge = Gauge('test_gauge', 'Test Gauge between 0 and 1')
7 train_gauge = Gauge('train_gauge', 'Train Gauge between 0 and 0.6')
8 test_hist = Histogram('test_hist', 'Test Histogram between 0 and 1', buckets=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0))
9 train_hist = Histogram('train_hist', 'Train Histogram between 0 and 0.6', buckets=(0.1, 0.2, 0.3, 0.4, 0.5, 0.6))
10
11 def emit_data():
12     """Emit fake data"""
13     time.sleep(5)
14     test_value = random.uniform(0, 1)
15     train_value = random.uniform(0, 0.6)
16     test_gauge.set(test_value)
17     train_gauge.set(train_value)
18
19     test_hist.observe(test_value)
20     train_hist.observe(train_value)
21
22
23
24 if __name__ == '__main__':
25     start_http_server(8000)
26     while True:
27         emit_data()
28
```

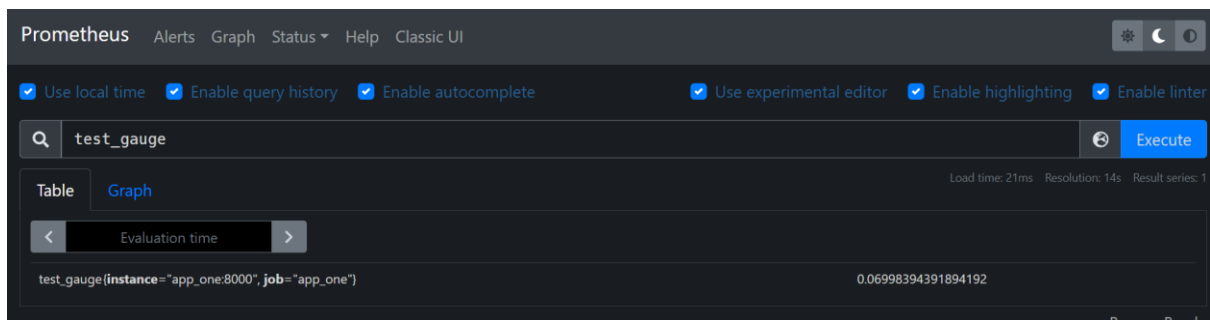
Rebuilding of app_one:

```
oadewusi@oluSec:~/aiops/Assignments/LAB2-Prometheus_Grafana$ docker compose build app_one
[+] Building 13.6s (13/13) FINISHED
=> [app_one internal] load build definition from Dockerfile
=> => transferring dockerfile: 495B
=> [app_one internal] load metadata for docker.io/library/python:3
=> [app_one auth] library/python:pull token for registry-1.docker.io
=> [app_one internal] load .dockerignore
=> => transferring context: 2B
=> [app_one 1/6] FROM docker.io/library/python:3@sha256:7859853e7607927aa1d1b1a5a2f9e580ac90c2b
=> [app_one internal] load build context
=> => transferring context: 925B
=> CACHED [app_one 2/6] RUN wget -O /usr/local/bin/dumb-init https://github.com/Yelp/dumb-init/
=> CACHED [app_one 3/6] RUN chmod +x /usr/local/bin/dumb-init
=> [app_one 4/6] COPY /app
```

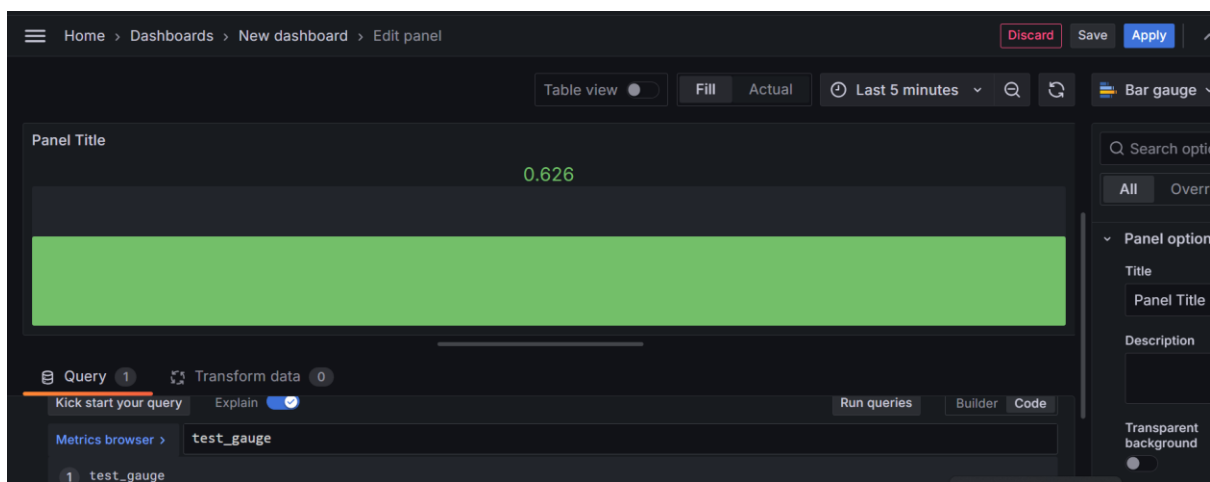
```
oadewusi@oluSec:~/aiops/Assignments/LAB2-Prometheus_Grafana$ docker compose up -d
[+] Running 8/8
✓ Container prometheus_node_exporter Running
✓ Container prometheus_push_gateway Running
✓ Container grafana Running
✓ Container app_two Running
✓ Container postgres Running
✓ Container postgres_exporter Running
✓ Container app_one Started
✓ Container prometheus Running
oadewusi@oluSec:~/aiops/Assignments/LAB2-Prometheus_Grafana$
```

New test metrics:

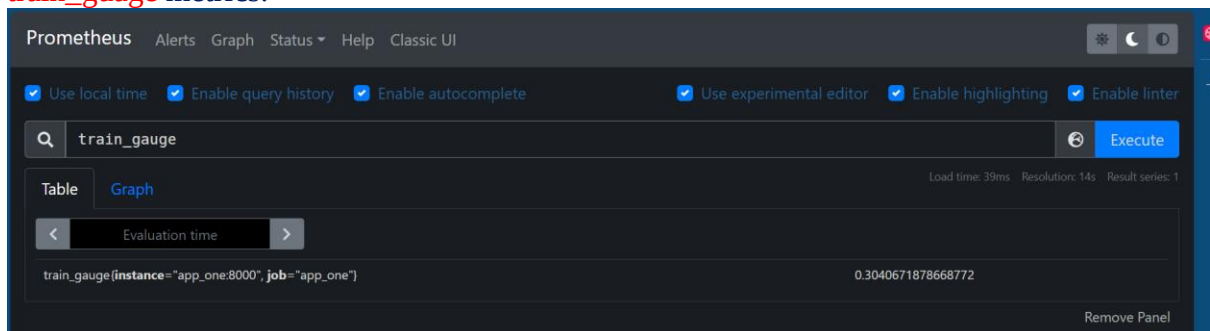
test_guage



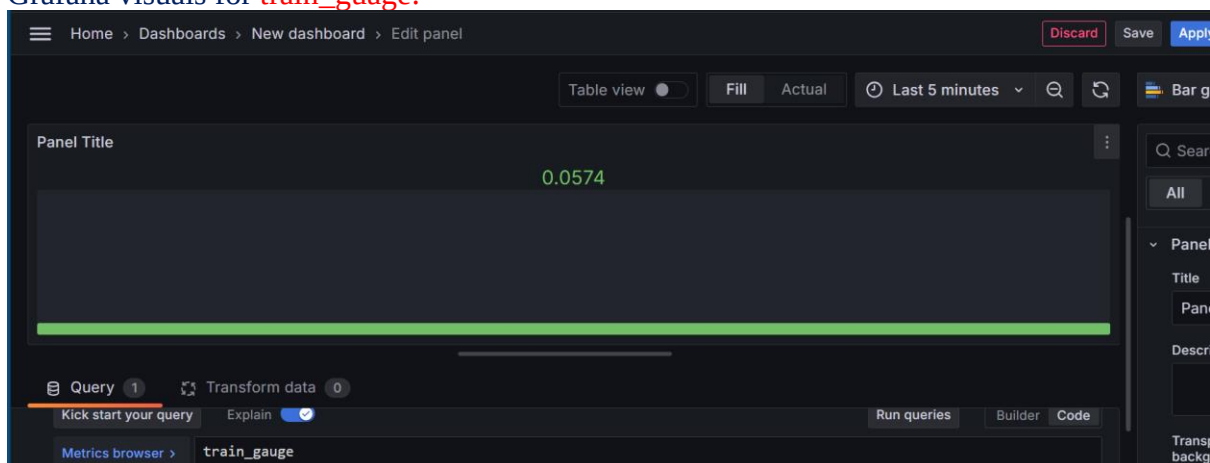
Grafana visuals for **test_gauge** metric:



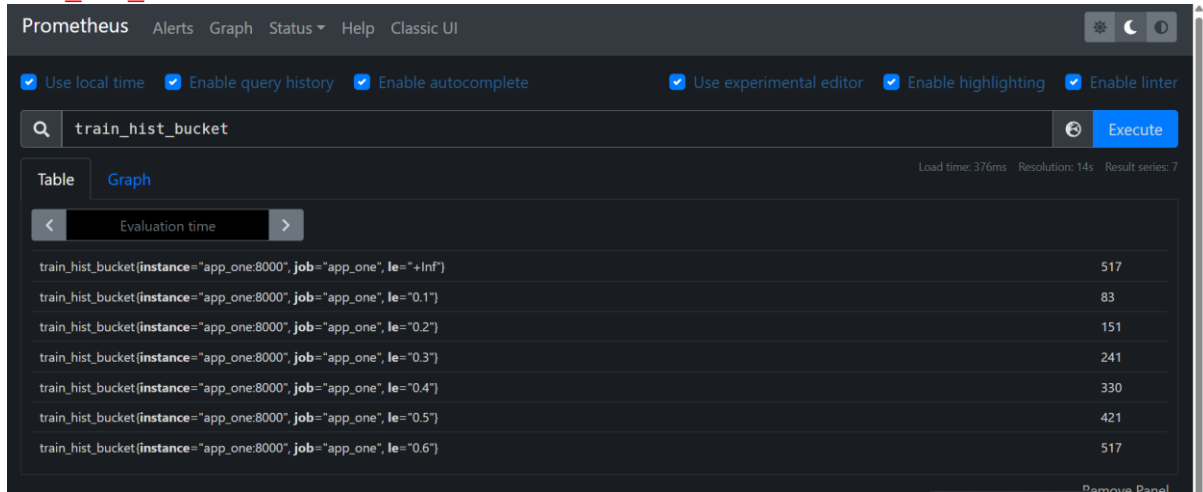
train_gauge metrics:



Grafana visuals for **train_gauge**:



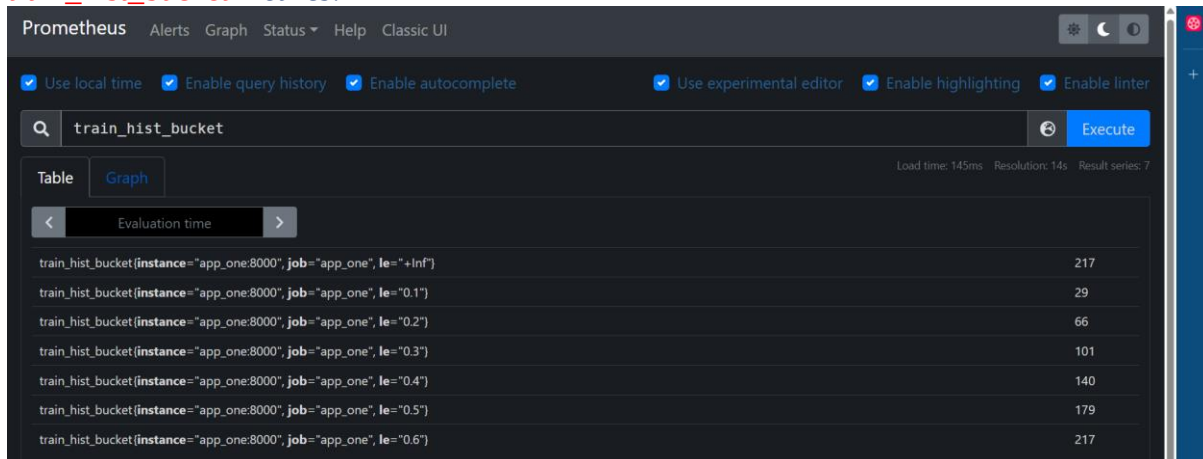
train_hist_bucket metrics:



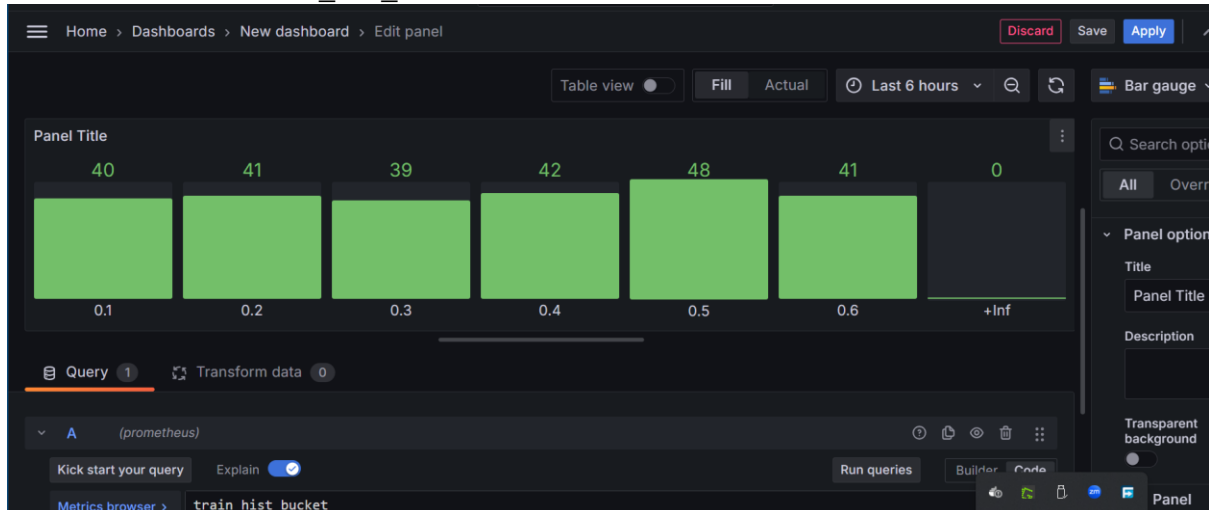
Grafana Visuals for train_hist_bucket:



train_hist_bucket metrics:



Grafana visuals for train_hist_buckets metrics:



Grafana visuals for the 4 new metrics combined:



Description of Docker Images and Running Containers

Docker Images:

Docker images are the templates used to create containers. These images are created using a dockerfile or a docker-compose.yml configuration file. When it being deployed, docker creates and manages multiple containers as a single application or service. It orchestrates the entire setup based on the configuration defined in the docker-compose.yml file. Each service defined in the yaml file corresponds to a docker image.

When executed with a docker-compose.yml configuration file, docker reads the docker-compose.yml file and creates the necessary containers, networks, volumes, and other resources defined in the file.

To see the created images and their details, I used the `docker images` command in the terminal. The `docker images` command shows a list of currently running images. Here's a typical output:

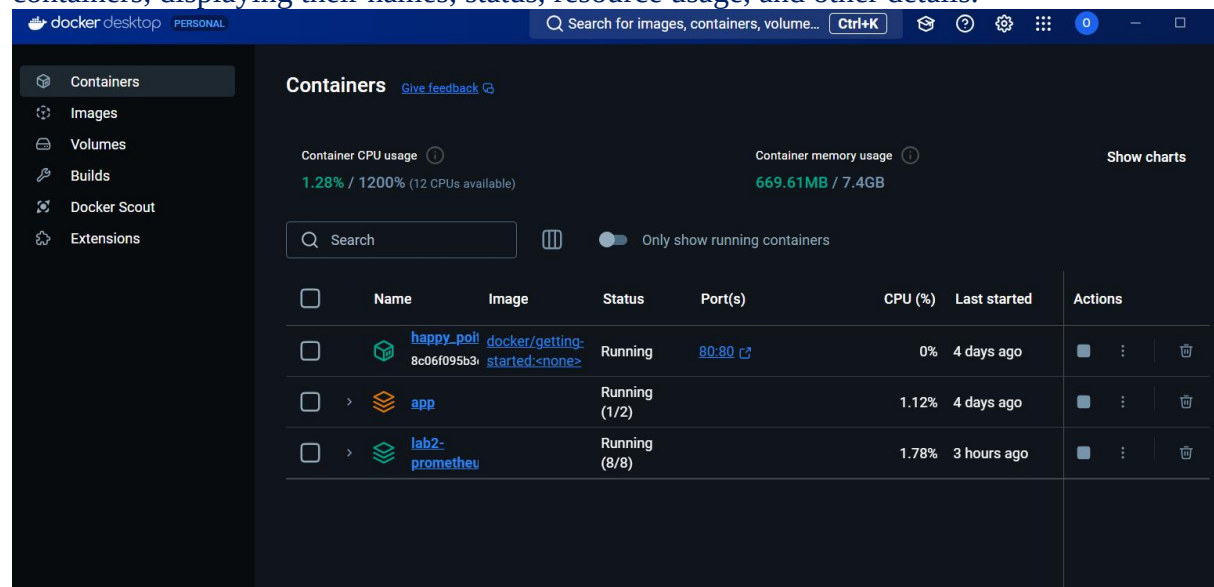
```
oadevusi@olusSec:~/aiops/Assignments/LAB2-Prometheus_Grafana$ docker images
REPOSITORY          TAG          IMAGE ID      CREATED      SIZE
lab2-prometheus_grafana-app_one    latest      ae159438d71d  2 hours ago  1.02GB
<none>                  <none>      93e872049a30  31 hours ago  1.02GB
lab2-prometheus_grafana-grafana    latest      f70290831e09  31 hours ago  478MB
getting-started          latest      9bd4dcb3cd81  4 days ago   218MB
oadevusi/getting-started          latest      9bd4dcb3cd81  4 days ago   218MB
lab2-prometheus_grafana-app_two    latest      2b20c0f3363c  9 days ago   14.6MB
lab2-prometheus_grafana-prometheus latest      cb8a81173aec  9 days ago   189MB
mysql                    8.0          307199bdf3e2  8 weeks ago  573MB
quay.io/prometheus/node-exporter   latest      71dc9668b154  2 months ago  23.3MB
prom/pushgateway          latest      ba433c5391e3  3 months ago  21.5MB
docker/getting-started          latest      3e4394f6b72f  21 months ago  47MB
postgres                  13.3        b2fcd079c1d4  3 years ago   315MB
wrouesnel/postgres_exporter        latest      9fe9d3d02141  3 years ago   88.7MB
```

Running Containers:

A running container can be viewed using the `docker ps` command in the terminal. This command will show you the list of all running containers, their names, assigned ports, and other information.

```
oadevusi@olusSec:~/aiops/Assignments/LAB2-Prometheus_Grafana$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED      STATUS      PORTS                               NAMES
ae55604e18c3   lab2-prometheus_grafana-app_one     "/usr/local/bin/dumb..." 2 hours ago  Up 2 hours  0.0.0.0:8000->8000/tcp             app_one
acb01a31934e   wrouesnel/postgres_exporter         "/postgres_exporter"      31 hours ago Up 31 hours  0.0.0.0:9187->9187/tcp             postgres_exporter
0f6f7f4e7701   lab2-prometheus_grafana-prometheus  "/bin/prometheus --c..." 31 hours ago Up 31 hours  0.0.0.0:9090->9090/tcp             prometheus
fbee51a6fc5d   lab2-prometheus_grafana-app_two     "/bin/sh -c /go/bin/..." 31 hours ago Up 31 hours  0.0.0.0:8001->8001/tcp             app_two
bf69fb3f9a18   postgres:13.3                       "docker-entrypoint.s..." 31 hours ago Up 31 hours  0.0.0.0:5432->5432/tcp             postgres
eef3d4e32c7    quay.io/prometheus/node-exporter     "/bin/node_exporter"      31 hours ago Up 31 hours  0.0.0.0:9100->9100/tcp             prometheus_node_exporter
b61223bde43b   prom/pushgateway                    "/bin/pushgateway"        31 hours ago Up 31 hours  0.0.0.0:9091->9091/tcp             prometheus_push_gateway
520b5141e954   lab2-prometheus_grafana-grafana     "/run.sh"                 31 hours ago Up 31 hours  0.0.0.0:3000->3000/tcp             grafana
b0462d8dccc3   mysql:8.0                           "docker-entrypoint.s..." 4 days ago   Up 4 days   3306/tcp, 33060/tcp              app-mysql-1
8c06f095b3c7   docker/getting-started              "/docker-entrypoint..." 4 days ago   Up 4 days   0.0.0.0:80->80/tcp               happy_poitras
```

The Docker Desktop also provides a visual representation of the image, volumes and running containers, displaying their names, status, resource usage, and other details.



Docker compose file explanation.

Services under the compose file:

- **app_one:**
 - container_name: "app_one"(name of the container)
 - -build: "./containers/app_one" (builds the image from this directory)
 - -ports: "8000:8000" (maps port 8000 of the host to port 8000 of the container)
- **app_two:**
 - -container_name "app_two"(name of the container)
 - -build: "./containers/app_two"(builds the image from this directory)
 - -ports: "8001:8001" (maps port 8001 of the host to port 8001 of the container)
 - -environment variable: DOCKER_NETWORK: push_gateway:9091 (indicates an environment variable possibly used to specify network-related configurations or endpoints within the service)
- **prometheus:**
 - -container_name:"prometheus"(name of the container)
 - -build: "./containers/prometheus"(builds the image from this directory)
 - -ports: "9090:9090" (maps port 9090 of the host to port 9090 of the container)
- **grafana:**
 - -container_name:"grafana"(name of the container)
 - -build: "./containers/grafana"(builds the image from this directory)
 - -ports: "3000:3000" (maps port 3000 of the host to port 3000 of the container)
- **node_exporter:**
 - -container_name:"prometheus_node_exporter" (name of the container)
 - -image: "quay.io/prometheus/node-exporter" (builds the image from this directory)
 - -ports: "9100:9100" (maps port 9100 of the host to port 9100 of the container used to export node metrics)
- **push_gateway:**
 - -container_name:"prometheus_push_gateway"(name of the container)
 - -image: "prom/pushgateway"(builds the image from this directory)
 - -ports: "9091:9091" (maps port 9091 of the host to port 9091 of the container used for metrics aggregation)
- **postgres:**
 - -container_name:"postgres"(name of the container)
 - -image: "postgres:13.3" (builds the image from this directory)
 - -Restart Policy: "always" (container will always restart if it stops)
 - -ports: "5432:5432" (maps port 5432 of the host to port 5432 of the container; PostgreSQL database)
 - -environment variables: POSTGRES_PASSWORD: example (sets the password for the PostgreSQL database)
- **postgres_exporter:**
 - -container_name: "postgres_exporter"
 - -image: "wrouesnel/postgres_exporter"
 - -ports: "9187:9187" (maps port 9187 of the host to port 9187 of the container; used for exporting PostgreSQL metrics)

- -environment variables: DATA_SOURCE_NAME:
postgresql://postgres:example@postgres:5432/postgres?sslmode=disable" (provides connection details to the PostgreSQL database)

Explanation of the Docker Network

By default, Docker Compose creates a single network for all containers defined in the docker-compose.yml file. This network allows containers to communicate with each other using their service names as hostnames. In this setup, containers like app_one, app_two, prometheus, grafana, and others are connected to the same default network. This network facilitates inter-container communication. The containers are attached to a **default bridge network**.